P1 实施指南：训练数据流加速系统 TrainCache

P1 实施指南：训练数据流加速系统（代号 TrainCache）— v3 重构版

> 2026-10-05 v3 重构（导师与学员讨论定稿）：本次重构修正了 v1 里的指标错位、对照组缺失、术语硬伤和过时的负载模型。项目的结构总览见 `03-项目结构定稿.md`（v3），本文是 P1 的详细实施指南。 配套文档： `07-开源贡献指南.md`（本项目回馈上游的目标已经改为 **Fluid**）。这份指南的目标和以前一样，希望你看完之后"照着就能开工"。

1. 项目介绍

电梯陈述：这个项目要做一套面向 K8s 训练 Job 的"预热-调度闭环"系统。工业界的现有方案把两件事情做得很松：数据预热靠人手动触发，缓存节点亲和只有粗粒度的调度提示。我们要把它们接成一个由调度决策驱动的闭环：控制器按照 Job 实际会消费的数据子集，以块级粒度向候选节点预热数据，再用细粒度的就绪度来门控或给作业打分，超时的时候允许降级。最后，我们用**端到端 makespan** 和真实部署起来的 Fluid/JuiceFS 同台跑分，量化这套方案在什么条件下能赢、在什么条件下反而会更慢。

三重定位（与 P2 同构，三者缺一不可）：

1. **对标者**：我们把 Fluid（Dataset/DataLoad/Runtime 加缓存亲和调度）和 JuiceFS 社区版当作一等对照组真实部署起来，在同样的负载下跑分，绝不打空气靶；
2. **测量者**：我们最看重的指标有两个，一个是 Job 从提交到跑起第一个 step 的时间，另一个是整个 epoch 的 makespan，预热时重复搬运的字节成本也要计入分母；我们允许阴性结论，并且在实验开始之前就把假设预注册下来；
3. **改进者**：现勘中确认下来的缺口，要用测量数据撑住，再向 **Fluid（Apache-2.0 许可、Go 语言，调度和预热正好落在它的代码域里）** 提 issue 和 PR，不能只停留在"我自己的仓库里有这个功能"。

这个项目明确不做什么：它不会再做一个分布式文件系统（我们不重造 JuiceFS 的 POSIX/FUSE 数据面）；它也不是一篇缓存算法论文（W-TinyLFU 在我们这里只充当工程对照，并不构成学术声称）；它全程都不需要 GPU——整个项目在 CPU 训练循环上就可以测，我们测的是 dataloader stall，跟 GPU 利用率没有关系。

2. 问题与动机（含 2026-10 生态现勘）

2.1 外部事实（2026-10-05 核实，这些事实构成立项的边界）

- **Fluid 的缓存亲和调度，本质上是"粗粒度的二进制判断加事后注入"**：mutating webhook 在 Pod 创建的时候读取它引用的 Dataset，然后往 Pod spec 里注入 `nodeAffinity`（ACK 增强文档给出的默认权重为：缓存节点 100、同可用区 50、同地域 20，给节点打标签还可以升级成硬亲和），另外还有 `RequireNodeWithFuse`（要求节点上已经存在 FUSE 客户端）与 `PreferNodesWithoutCache`（把不消费该数据集的 Pod 引导到别处）两个策略；如果 Pod 自己带了 affinity 或 nodeSelector，webhook 就会跳

过注入。它能回答的问题只有"节点上**有没有**这个 Dataset 的 runtime/缓存"，**完全不知道缓存了多少比例、具体缓存了哪些块**。

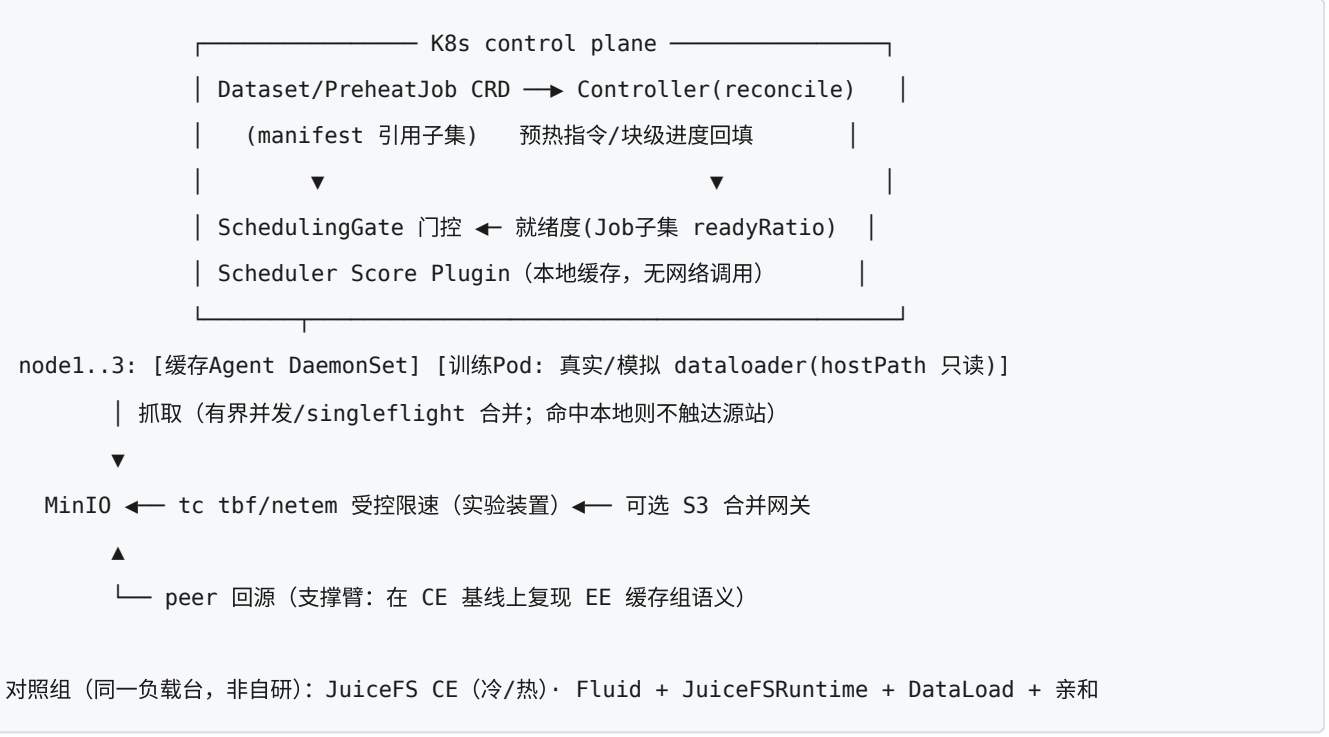
- **预热（DataLoad）是一个独立的、需要手动创建的 CRD**，它和调度决策之间没有闭环：DataLoad 不会因为"某个 Job 马上要被调度"而触发，调度器也不会根据 DataLoad 的块级进度来做门控。开源 Fluid 与 ACK 托管增强版（ack-fluid，就是那套用 ConfigMap 配置权重的增强能力）之间的能力边界，你必须在现勘阶段划清楚。
- **JuiceFS 社区版 v1.3.1 采用 Apache-2.0 许可**（早期"AGPLv3"的说法已经过时）：它支持 4MiB 块、warmup、`--prefetch` 预读和单机本地缓存；**分布式缓存组（一致性哈希环、peer 之间共享块）、IORPC、RDMA 都属于企业版能力**（EE 5.2/5.3 及以上版本）。本项目使用的 4MiB 块大小与社区版默认值对齐，这一点会作为报告注脚写明。
- **2026 年训练数据加载的主流形态**：webdataset tar 分片、Mosaic MDS 这类百 MB 级分片，配合确定性 shuffle 做流式读取；百万小文件的 ImageNet 模式现在只属于元数据压力场景，已经不再是 LLM 和大模型训练的典型 I/O 形态。

2.2 仍然成立的问题（这就是本项目要钻的缝）

1. **预热与调度之间没有闭环**：工业方案的做法是"先手动建一个 DataLoad，然后靠二进制亲和碰运气"，这两步之间缺少一条完整的控制回路——"Job 即将被调度 → 判断该往哪几个候选节点预热它需要的那些块 → 用块级就绪度决定何时放行、如何打分 → 超时之后怎么降级"；
2. **就绪度的粒度对不上**：节点上"有缓存 runtime"和"这个 Job 要读的分片 90% 都在本地"完全是两回事；在多 rank、多分片训练下，Job 往往只消费数据集的一个子集；
3. **预热的成本没有人记账**：往 2 到 3 个候选节点重复预热，要付出 2 到 3 倍的字节与带宽代价，但这笔代价在什么情况下值得花，目前没有公开的曲线。

立项闸门（规矩和 P2 一样）：§2.1 对缺口的判断，必须在阶段 0 由**开源 Fluid 的实测取证**来确认（包括 webhook 注入了什么内容、DataLoad 与调度之间的时序关系）；如果现勘发现开源 Fluid 已经提供了块级进度门控，叙事就收缩为"复现与量化"，不许硬造缺口。

3. 架构总览



数据面约定（保留 v1 的决策）：我们采用 preheat 模型——Agent 把数据块落到 `/var/cache/traincache/<ds>/blocks/`，训练 Pod 通过 hostPath PV 只读挂载这个目录，读数据时直接走内核的 page cache，**Agent 完全不参与读 I/O 路径**，所以读路径上没有任何额外延迟。代价是读侧的信号只能靠旁路采集（见 §5.4）。FUSE read-through 降级为可选的 stretch 目标（Fluid 生态里已经有成熟的 FUSE/CSI，我们不重造）。

4. 阶段 0：现勘（与使用阶段合并，1–2 周，闸门制）

4.0 可行性探针（最先做，2–3 天，在写任何控制器代码之前）

这一步的目的，是先用几十行脚本证明实验方法论站得住脚——如果实验装置本身造不出可信的差异，三个月后再回头改设计，赔上的就是整个项目。这三天是整个 P1 性价比最高的三天。

- **探针 A：验证冷热差稳定且可复现**。准备同一批百 MB 分片，用 `tc qdisc ... tbf rate 100mbit burst ... netem delay 20ms` 挂在 Agent 到 MinIO 的路径上；冷读（先执行 `sync; echo 3 > drop_caches` 并清空本地缓存，读请求走 tc 链路）与热读（命中本地缓存，不经过 tc）各跑至少 7 次。
  - **绿灯标准**：冷读与热读的中位延迟相差至少 5 倍（或者中位吞吐差一个数量级）；两组的 bootstrap 95% 置信区间**互不重叠**；组内变异系数 CV 不超过 15%；
  - **如果亮了红灯，正确的做法是去排查装置本身，不要硬调参数去凑数字**：tc 是不是挂在了正确的网卡和出口上（用 `tc -s qdisc` 观察计数有没有在涨）、热读是不是真的零网络流量（用 `ss` 或 tcpdump 验证没有建连）、单块磁盘自己是不是已经成了瓶颈、MinIO 自己的 page cache 会不会制造出"假冷读"；

- **探针 B：验证 stall 指标能打出来**。搭一个最小的 CPU 版 PyTorch DataLoader（用 webdataset 读真正的 JPEG tar，`num_workers` 取 0/4/8 三档，不挂模型或只挂极小的矩阵乘），在训练循环里打点记录 `wait_time/step_time`。
  - **绿灯标准**：冷态下 stall 占比稳定高于 40%，热态下低于 15%；增加 workers 数量之后冷态 stall 单调下降（打点噪声造不出这种单调下降，所以它能证明我们测到的是真实的 I/O 饥饿）；
- **硬性时间盒为 3 天**。如果到第 3 天晚上仍然亮红灯，就停止开工，复盘根因（必要时上外接 SSD，或者提前租两台带独立数据盘的 CPU 云主机），**绝不带着毛病进入步骤 1**。

动作清单：

1. 搭好版本不低于 1.26 的 kind 集群（这是 PodSchedulingGate 的前置要求），在上面部署 MinIO、**JuiceFS 社区版（本地缓存模式）和开源 Fluid + JuiceFSRuntime**；
2. 跑通官方 demo：创建 Dataset、用 DataLoad 手动预热、启动带 dataset 标签的训练 Pod；抓取并归档 **webhook 实际注入的 Pod spec**（nodeAffinity 权重、FUSE 注入情况），以及 DataLoad 与 Pod 调度之间的**时序关系**（Pod 会不会等 DataLoad 完成？还是先被调度、再等数据？）；
3. 整理一张能力边界表，把开源 Fluid、ACK 文档中的增强能力、JuiceFS 社区版、JuiceFS 企业版（缓存组/RDMA）四方放在一起对比；每一个判断都要附上版本号和证据（kubectl 输出、文档链接或截图）；
4. 用 JuiceFS 的 `warmup` 完成第一次"对手热读"基线，记录版本号与挂载参数；
5. 产出 **《P1 对手现勘报告》**（这份报告和 P2 的现勘报告合并为同一时期的总交付物）。

闸门产出：①如果缺口被证据确认，核心 1 就按"闭环"叙事推进；②如果缺口不成立，核心 1 改为"Fluid 机制的独立复现与参数量化"，不许带着假设开工。

5. 基座设计

5.1 CRD 设计

```
apiVersion: traincache.io/v1alpha1
kind: Dataset
spec:
  source: { endpoint: http://minio:9000, bucket: datasets, prefix: shards/ }
  chunking: { strategy: fixed, chunkSize: 4MiB }   # 与 JuiceFS 默认 block 对齐
  index: { algo: path-index }        # 索引层：路径寻址（见 5.2）
  storage: { digest: xxh3-128, dedup: true }        # 存储层：CAS
  version: v5                        # 版本前缀目录，不做自研失效协议（见 §9.3）
status:
  totalChunks: 262144
  nodes:                             # 粗粒度状态，细清单不下发 etcd
    - { node: node1, readyRatio: 0.98, bytes: 1.0Ti, inflightBytes: 64Gi }
  conditions: [{ type: Ready, status: "False", reason: Preheating }]
```

- **etcd 对象有 1.5MB 的上限**：块级清单放在对象存储上的 index shard 里（每个 shard 不超过 10 万条），CRD status 只存每个节点的 readyRatio 加上 shard 位图摘要（roaring bitmap 序列化之后的摘要哈希）；调度插件需要精确清单的时候，再按需拉取对应的 shard；
- **Job 的子集引用**：训练 Job（尤其是多 rank 的 Job）通过 label 或 annotation 声明自己要消费的 manifest 子集（分片列表）；就绪度一律按**子集**计算，不按整个数据集计算；
- `version` 字段就是源前缀的代次（类似 `shards/v5/` 这样的前缀目录）；index 构建支持增量：ETag 没有变化的 shard 直接跳过，避免对百万对象做全量 List。

5.2 双层标识模型（修正 v1 的术语硬伤）

v1 里的 `块ID = xxh3(path+index)` 实际上是**路径寻址加校验**，谈不上内容寻址——v3 明确把它拆成两层：

- **索引层（决定取什么）**：manifest 条目键为 `(path, chunkIndex)`，采用路径寻址。它不承诺跨路径去重；
- **存储层（决定怎么落盘）**：块在预热下载的时候**本来就要流过内存**，我们趁这个机会流式计算 xxh3-128 摘要（速度大约 10GB/s 每核，不产生额外 I/O）作为存储键，落盘到 `blocks/ab/<contentHash>.blk`，并维护一份 refcount 索引。这样一来：
  - 读回校验就成了内置能力（块头的元信息里用 64B 记录大小和摘要，不搞 sidecar 文件）；
  - **安全去重也自动成立**：不同路径下的相同内容、多个分片的公共头部都只占一份空间，只有 refcount 归零的块才进入可驱逐集合；
- 措辞统一为"**路径索引 + 内容摘要存储（CAS 存储层，可选去重）**"。面试时你可以这样讲："寻址回答的是'我要什么'，摘要是块落盘之后才算出来的，没法反推出路径——我在 v1 里把这两层混为一谈了，后来纠正了过来。"
- 哈希选型上，内部信任边界里我们用 xxh3-128 换取性能，全量 SHA-256 校验保留为开关；碰撞概率与信任边界之间的权衡，你要能讲清楚。

5.3 缓存 Agent（Go，保留 v1 的工程设计）

并发模型：

```
抓取管线 = workqueue(块, 优先级) → errgroup + 按字节加权信号量 → 分块写盘
限制 bytes-in-flight（默认 512MiB）而非 goroutine 数：防小文件 goroutine 爆炸、大文件 OOM。
```

- HTTP 层的 `MaxConnsPerHost`（默认 64）与 MinIO 的连接池对齐；抓取优先级从高到低为：显式预热、邻域预取、后台补全；
- **原子可见性**：先写 tmp 文件再 `rename()`；缓存块不逐块 fsync（数据丢了可以重新拉取，掉电丢块的代价就是重新下载）——这个取舍和存储系统的直觉相反，但它是正确的；
- **驱逐策略**：采用双水位（占用到 85% 触发驱逐，一路回收到 75%），按 LRU 顺序驱逐，同时用 refcount 把正在读的块钉住；`unlink` 一个已经打开的文件是安全的（拿着 fd 的读者仍然可以读），所以驱逐不需要等读者结束；配额按 Dataset 分组；
- **启动重扫描**：Agent 重启之后扫描 blocks 目录重建 CAS 索引，对块头抽样校验 1%，全量校验放在后台以低优先级进行——这是混沌韧性（§8.4）的地基。

5.4 读侧信号与命中率口径（保留，诚实标注）

- 用 inotify 监听缓存目录的 `IN_OPEN` 事件（做去抖：同一个 pid 多次打开同一个文件合并成一次，采样率可配），由此得到文件的打开频率和顺序；
- **命中率口径要写进报告**：消费者侧——由插了桩的 dataloader 上报它打开的块集合，再和本地块集合求交；我们**不声称 Agent 在读路径上做了精确记账**（preheat 模型做不到这一点）；更进阶的替代方案是 eBPF（那是 S3 项目的内容）。

6. 核心 1：预热-调度闭环（项目灵魂）

我们把 v1 的增量 1（gate）和增量 2（预热状态机）合并成同一个控制回路。

6.1 控制回路

```
Job 创建（声明 Dataset + 消费子集 + 门控模式注解）
  → Controller 注入 SchedulingGate: traincache.io/preheat-pending
  → 按放置策略选候选节点（fanout=1 / 2-3 / 全部，本身是实验变量）
  → 向候选 Agent 下发"子集块清单"预热任务（字节预算 + 速率上限）
  → Agent 块级进度回填 status
  → 子集 readyRatio ≥ 阈值（默认 0.9）→ 摘除 gate
     或总超时（默认 5m，注解可覆盖）→ 降级：去 gate/去亲和正常调度
  → Score plugin 按"节点持有该 Job 子集块比例"打分 → bind
```

实现时要注意这些点（保留 v1 踩过的教训）：

- gate 的注入和摘除必须幂等，再加 finalizer 防止产生孤儿对象；**k8s 版本不低于 1.26**，kind 镜像版本在阶段 0 确认；
- **Score plugin 内部不允许做网络调用**：用 informer watch 聚合 status，配合本地缓存加 1 秒 TTL——把 scheduler 打挂是调度器扩展里的经典坑；
- 打分以 `score(node) = readyRatio_subset` 为主，节点空闲磁盘和在途预热负载作为次要排序依据；门控模式分两种：`required`（硬等，达不到阈值就只能等超时降级）与 `preferred`（软门控，超时立即放行）；
- 降级语义要设计成"去掉约束继续调度"，不能直接报错；降级时写入 Condition=Degraded、记录 Event 并上报 Prometheus 指标；数据源不可用的时候有总超时兜底，**Job 永远不会因为预热问题而死锁在 Pending 状态**；
- 预热速率受预算约束（默认不超过节点可观测空闲带宽的一定比例），防止预热点火的时候挤压在线 Job。

6.2 预注册假设与盈亏平衡旗舰图

v1 的致命问题在于：它只度量 gate 摘除之后的等待时间，却不把预热成本记入账内。v3 把头号指标改为：

类别	指标
效果	**Job 从提交到跑起第一个 step 的时间**；**整个 epoch 的 makespan**；dataloader stall 占比（等待时间除以总时长，在 CPU 训练循环里直接打点，不需要 GPU）
成本	预热总字节数 × fanout（重复预热的倍数）；源站出口字节数；预热带宽对在线流量的挤压程度（在跑 Job 受到的 stall 扰动）
过程	各候选节点 readyRatio 的时间序列；降级触发率；门控等待时长

假设要在实验开始之前写死（预注册，最后数字长什么样就报什么样）：

- 我们**预期门控策略会赢**的场景：①多 epoch、多轮重复训练；②数据集大于缓存容量，但热子集稳定；③多个 Job 共享同一份数据；④源站很慢或带宽很贵（tc 的高延迟限速档）；
- 我们**预期门控策略可能会输**的场景：①只跑一次全量顺序 epoch（预热和训练读的是同一遍数据，门控只会增加串行化）；②缓存容量远大于数据集；③源站极快；④预热频繁超时（这种情况下预热还不如不预）；
- **旗舰图是一张盈亏平衡曲面**：以源站带宽、数据集/缓存容量比、fanout 三个维度扫点（也可以固定一维画出二维曲线），标出"闭环门控的 makespan 减去自由调度的 makespan"的正负区间。这张图即使大半区域都是负的，也算成功——**因为它回答了"这东西什么时候不该用"，而这个问题恰恰是面试官最想听的工程判断**。
- 配套的次旗舰图：fanout 取 1/2/3 时，**makespan 收益与重复字节成本**的 Pareto 散点图。

7. 核心 2：欠配缓存下的准入与预取（场景修正，接受阴性）

7.1 为什么 v1 的 W-TinyLFU 旗舰图可能是一条平线

全量顺序 epoch 加上缓存容量足够大时，LRU、LFU 和任何准入策略的命中率都会趋同——准入策略之间的差异，只有在**缓存欠配、访问倾斜、热子集被重复消费、多租户互相挤占**四个条件同时成立时才会显现。所以这个实验必须主动构造上述场景，不能在顺序扫描上干巴巴地比较三个算法。

7.2 实验设计

- **多 Job 倾斜 trace（负载 C 轨，§10.1）**：让 Job A 反复读取 10% 的热分片（模拟多轮微调和重复 epoch），Job B 顺序扫描全量数据（流寇式扫描），缓存容量设为数据集的 30%；
- 对照组：在 LRU、LFU、W-TinyLFU 三种策略下，分别记录 **Job A 的命中率与 stall 占比**，以及扫描作业把热集驱逐出去的次数；
- W-TinyLFU 的实现（保留 v1 规格）：window（SLRU 的 probationary/protected 两段）加 count-min sketch（4 行、宽度取缓存块数的 10 倍、3bit 饱和计数）加 doorkeeper bloom；准入规则是 `freq(candidate) > freq(victim)` 时才换入；定期做 halving；参数（window 占比、halving 周期）扫点画出敏感性图；

- **自适应预取**：inotify 观测到块 i 被打开，就抓取后继的 K=8 个块；如果预取命中率低于阈值，就把 K 降下来；
- **允许得到的结论**：如果全量顺序场景下三种策略没有差异，而倾斜场景下有差异，那就如实写"准入策略对单 Job 的顺序训练并不关键，真正关键的是预取调度与跨租户保护"。这是一个成熟的阴性发现，算不上实验失败。

8. 支撑臂设计

8.1 S3 合并回源网关（保留 v1）

- 用 `singleflight` 按对象 key 合并并发的 GET 请求；大对象通过 `io.Pipe` 或 splice 流式扇出，禁止整块缓冲之后再分发；
- AIMD 背压：当 MinIO 的 P99 延迟上升或出现 5xx 时，令牌桶减半，平稳期再线性恢复；并发上限本身就是"对中心存储的礼貌"；
- 用 `sync.Pool` 复用块级 buffer，并通过 `GODEBUG=gctrace=1` 对比每字节分配数和 GC pause（作为 Go GC 调优的热身）。

8.2 P2P 块共享：在 CE 基线上复现 EE"缓存组"语义（定位重写）

v1 的增量 5 只是一个普通的 P2P 实现；v3 给它找了明确的对标对象：**JuiceFS 企业版的分布式缓存组（用一致性哈希环定位块、peer 回源、节点增删时迁移）**。我们要用 Go 自研组件，在只有单机缓存的社区版基线上把这套语义做出来：

- 节点发现直接复用 CRD status：每个节点上报自己持有的块 shard 位图；一致性哈希决定每个块的"老家节点"；
- 拉取顺序：优先找持有该块的 peer，限制并发向 2–3 个 peer 多源拉取，peer 都没有再回源；peer 请求失败就立刻穿透回源（这与企业版文档描述的故障语义一致：peer 故障并不致命）；
- **全局带宽预算（P2C 思想，保留）**：每个节点一个令牌桶，peer 流量和 origin 流量共用，origin 保底占 20%，防止 P2P 把训练流量挤掉；
- 传输层抽象为 `Transport` 接口：gRPC stream 是默认实现；**RDMA 作为实验臂插在接口后面**（§9.1），永远不进主路径；
- 对照指标：源站出口字节数、多节点冷启动的 makespan；我们的性能数字和企业版官方口径只做量级参照，并且明确标注两边条件不同、不能直接比较。

8.3 Checkpoint 异步 sidecar（修正 v1 的语义错误）

- **写路径（保留）**：trainer 写到共享的 emptyDir，sidecar 通过 fsnotify 感知，然后切成 16MB 定长分片、逐片计算 xxh3、用 8 个并发做 multipart 上传，**最后写 manifest 对象作为发布点**（这样 S3 上永远只能看到完整的 checkpoint）；multipart 的 ETag 并不是 md5，校验以 manifest 里的逐片哈希为准；
- **恢复路径（修正）**：v1 里"权重先到、模型先起、optimizer state 后到"的说法，对**训练恢复场景是错误的**——optimizer state（比如 Adam 动量）缺失时继续 stepping，会改变收敛轨迹。正确的做法要区分两个场景：
  - **推理或 eval**：权重到位就能用，这个说法成立；

- **训练恢复**：权重和 optimizer state **都完整之后，才允许跑第一个 step**；sidecar 的真正价值在于"异步分片上传不阻塞训练循环，恢复时用 Range GET 并发拉取，加载与初始化重叠起来以缩短 stall"，它省不下状态本身；

- 指标：checkpoint 写入对训练 step 的扰动、恢复总时长的拆解（拉取、加载、与初始化重叠的部分）。

8.4 混沌套件（保留 v1 四件套）

注入	手段	期望
Agent kill -9	让进程崩溃再重启	重扫描后恢复索引，没有数据错误
网络劣化	tc/netem（仅在特权测试容器里使用）	背压生效，P99 恶化但有界
位翻转	通过测试钩子篡改块字节	校验失败 → 重新拉取 → 有指标记录
数据源宕机	把 MinIO 缩容到 0	门控超时后降级，Job 不会卡死

9. 调研/实验臂（只做实验或只写文档，绝不进主路径）

9.1 RDMA 传输臂（无硬件条件下的可行路径，承接你的个人兴趣）

- **在笔记本上跑真正的 verbs 代码**：Soft-RoCE（内核模块 `rdma_rxe`）提供完整的 libibverbs/librdmacm 语义（QP 状态机、MR 注册、RC/UD），API 和真卡完全一致；
- **铁律：Soft-RoCE 跑出来的性能数字一律不进报告**（它要绕内核做 UDP 封装，常常比 TCP 还慢），我们只用它验证功能正确性、连接管理和故障语义；kind 里面跑 rxe 有网络命名空间的坑，开发阶段先在宿主机上做，或者用 `--network host` 的特权容器；
- **真值按小时租**：阿里云 eRDMA（ERI）在普通的 c7/g7/r7 规格机器上就能挂载，你可以租两台同可用区的小规格 CPU 实例对测一个下午（花费大约几十元），拿到的数字要标注为一次性校准环境的结果；
- 实验内容：4MiB 块在 TCP 与 RDMA 下的吞吐、CPU 占用、拷贝路径对比；MR 注册开销与 MR cache；**不要通过 cgo 把 libibverbs 链进 Go 的热进程**——用一个 C++ 独立传输组件加一层薄薄的 C ABI（这条能力线和 S2 合流）；
- 位置：作为 §8.2 中 `Transport` 的实现 B，排在 gRPC 基线之后，永远是实验臂。

9.2 io_uring / S2（把收益预期预先诚实化）

4MiB 的大块经 HTTP 抓取时，整个过程受网络限速，本地写路径省下的 syscall 对吞吐的贡献预期接近于零；io_uring 真正的战场是**小文件高 IOPS 臂（负载 A 轨）**。报告里正确的写法是"syscall 数量大幅下降、大块吞吐不变并解释原因、小文件臂 IOPS 提升 X"。O_DIRECT 的三重对齐（偏移、长度、buffer 地址）、注册缓冲以及 fio 校准都照做。

9.3 版本化（v1 增量 7 砍掉自研协议）

版本前缀目录（如 `datasets/imagenet/v5/…`）天然就消解了失效问题，版本的寻址边界就是失效边界。我们只写一页 ADR 说明"为什么不自己造一套失效协议"，测试只覆盖版本切换、rename、重新提交这几条路径。GPU 侧技术（DALI/GDS/CUDA Stream）的知识边界见 §9.4。

9.4 GPU 侧技术的知识边界（DALI / GDS / CUDA Stream：只调研、写半页笔记、不动手）

一个训练 step 的数据路径与归属划分：

```
对象存储 →[网络]→ 节点本地盘 → page cache → DataLoader 解码/增强 → pinned memory →[PCIe H2D]→ GPU 显存
→ 计算
  t1 跨网络取数            t2 本地读      t3 CPU 解码      t4 H2D 拷贝（CUDA stream 可与 t5 计算重叠）
t5
  百 ms~秒级（冷读可分钟级）   ms/十 ms       十~百 ms          亚 ms~ms（默认已与计算重叠）
└──────────────── P1 认领：让数据在 t1 之前就位于本地盘（分布式系统/调度问题）────────────────┘
                                          └──── P1 边界之后：单机训练运行时（框架/CUDA 问题）────┘
```

- **DALI** 负责 GPU 侧的图像解码与增强，消灭 t3；**GDS（GPUDirect Storage）** 让数据从 NVMe 直接 DMA 到显存，绕过 CPU bounce buffer 与 page cache，优化 t2 到 t4 之间毫秒级的拷贝，但它需要文件系统、硬件、驱动全链路支持，条件不具备时会自动回退到兼容路径；**CUDA Stream** 让 t4 与 t5 重叠（在 DataLoader prefetch 下本来就重叠得不错）；
- **为什么我们不动手做**：①岗位分工——t1 是分布式系统问题（Go、调度、缓存，Fluid 和 JuiceFS 全部止步于"文件在本地 POSIX 路径就绪"），t2–t4 属于训练框架与 CUDA 问题（Python/C++，对标的是 DALI 和 NVIDIA），只要越界一层，五组对照与上游叙事就会断裂，作品的身份也会从平台项目变成半成品框架；②量级——t1 通常比 t4 大 2 到 3 个数量级；③真值——t2–t4 的性能结论必须在真 GPU 上才能得到，游戏本测不了，拿不出手的数字当不成弹药；
- **有两个边界题你必须能在白板上讲清楚**：
  1. **"训练用了 DALI，P1 还有价值吗？"**——DALI 只能消灭 t3 的解码等待，消灭不了跨网络取 shard 的 t1，DALI 自己也得先读到原始分片；所以"P1 让分片在本地就绪"这个价值依然成立，但"解码不再是瓶颈"会让收益区间收窄，这一点要作为盈亏曲面的讨论变量诚实标注；
  2. **"GDS 绕过 page cache，你的缓存命中模型怎么办？"**——GDS 绕过的是内核 page cache，绕不过本地盘；P1 的核心价值是"数据在本地且预热已经完成"，GDS 从本地 NVMe 直传反而更依赖本地就绪；你需要弄懂的是它的部署约束（文件系统、硬件、驱动支持与回退路径），不需要把它做进来；
- **交付物**：阶段 0 同期写半页笔记（包含本路径图和这两个问题的答案）；面试口径固定为"我没有动手做，但我知道它在流水线上和我的系统怎么接力"。

10. Benchmark 方法学（可复现纪律）

10.1 三轨负载

轨	形态	定位
A 元数据压力臂	100 万个 4KB 小文件（脚本生成，参数可调）	模拟元数据风暴和小文件 IOPS，是 io_uring 的战场；**不再自称训练主流**

轨	形态	定位
B 真实分片主臂	webdataset tar 或 Mosaic MDS，百 MB 分片加**真实 PyTorch DataLoader**，配合确定性 shuffle；**脚本合成真实 JPEG 编码的分片**（见下）	对应 2026 年训练的真实形态，所有旗舰数字都以这一臂为准，前移到步骤 5
C 多 Job 倾斜臂	Job A 反复读 10% 热分片，Job B 全量扫描，缓存欠配到 30%	核心 2 专用

模拟器只用来做参数扫点；旗舰数字必须有真实 dataloader 复跑。CPU 训练循环（不挂模型或只挂极小模型）以 dataloader stall 占比作为代理指标。

B 轨数据合成纪律（零下载、真解码）：我们不用纯随机 tensor（那样没有解码成本，被人追问"为什么不真实"时会答不上来），也不去下载真实的 ImageNet（带宽、许可、时间都不允许）；正确的做法是用脚本批量生成随机像素图像，做**真实的 JPEG 编码**（PIL 或 turbojpeg，质量和分辨率参数化），再按 webdataset 规范打成百 MB 的 tar 分片，固定随机种子，保证一键可以再生成。这样解码是真的、CPU 压力是真的、shuffle 也是真的；报告里要注明"合成数据，I/O 与解码路径真实，语义内容随机"，并把"合成图的解码成本分布可能不同于真实 ImageNet"列为外推威胁，有条件的话可以选一次小样本真实数据做 sanity check。

10.2 在笔记本上造出真实的冷热差（关键实验装置）

单机加一块物理盘，天然不存在"本地 NVMe 对比网络取数"的速度差。解决办法是**用 `tc qdisc tbf/netem` 对 Agent 到 MinIO 的源站路径做受控限速和延迟注入**（比如 100Mbps、20ms 这一档，并通过扫点模拟不同质量的云存储）；命中本地缓存的读请求根本不会走这条路径。这样一来，命中与未命中之间的延迟差既真实又可控，限速参数就成了实验自变量。

10.3 对照组矩阵（五组，共用同一条负载轨迹）

#	条件	说明
1	直读 MinIO	零缓存基线
2	JuiceFS CE 冷读	首次访问，本地缓存为空
3	JuiceFS CE warmup 热读	预热完成之后再读
4	**Fluid + JuiceFSRuntime + DataLoad + 亲和调度**	最强的工业基线，严格按官方姿势操作并记录全部 yaml
5	TrainCache	required 门控、preferred 门控、超时降级三种策略分别跑

五组实验共用一份固定随机种子的请求轨迹文件；跑在同一台机器上，遵守同一套冷态规程。**即使跑输第 4 组，也是一份合格的报告**——学习项目最大的原罪是不敢上秤，输了并不丢人。

10.4 硬纪律

1. 每组实验**至少跑 7 次取中位数，并报告 bootstrap 95% CI**；只跑 3 次取中位数的结果不进报告；

2. 每轮实验之间清页缓存（`sync; echo 3 > drop_caches`）并重启相关进程；MinIO 的数据目录与缓存目录尽量放在不同盘上（外接 SSD），实在分不开，就在报告里把共享盘列为已知的混杂因素；
3. 环境完全锁定：镜像 digest、JuiceFS/Fluid 版本、tc 参数、负载配置、随机种子全部入库，做到 `make bench COND=xxx` 一键复现；
4. 先写预注册假设（§6.2），禁止看到结果之后再调参"做出"目标数字；目标百分比（比如下降 60%）这类说法从正文里删掉，只保留假设中的预期方向。

11. 项目优点（面试时可以讲）

1. 完整的 Operator 工程：CRD（在 etcd 尺寸约束下的 shard 状态设计）、幂等 reconcile、condition/finalizer；
2. 调度器框架的全链路：SchedulingGate 加 Score（无网络调用的本地缓存范式）、块级子集就绪度、超时降级与死锁防护；
3. **"预热-调度闭环"是经过现勘取证的工业空白点，还配了一张盈亏平衡曲面——包含阴性区间的诚实量化**；
4. 双层标识模型（路径索引加 CAS 存储层）带来的内置校验与安全去重，以及"纠正自己术语错误"的真实设计过程；
5. 把缓存准入放在**正确的场景**下对照（欠配、倾斜、多租户），并且能讲清楚它在什么时候无效；
6. I/O 与分布式方面的硬货：singleflight 合并加 AIMD 背压、P2C 带宽预算、CAS 下 unlink/rename 的内核语义、混沌四件套；
7. 与真实的 Fluid/JuiceFS 做了五组对照，改进最终回馈给 Fluid——第三方背书是简历上信用最高的东西。

12. 诚实清单（不足之处主动说）

- 3 节点的 kind 集群只能用于机制验证和相对趋势分析，不做千卡规模的外推（用瓶颈模型讨论外推时要标注假设）；
- 单物理盘上的冷热差是用 tc 受控限速造出来的，绝对带宽数字最终要靠一次双盘云主机校准来替换；
- preheat 模型的命中率是旁路口径（inotify 加消费者打点），并不是读路径上的精确记账；
- 核心 1 的"空白"判断以阶段 0 的现勘为准，如果闸门反转，叙事就收缩，不硬撞；
- 没有真 RDMA/GDS 的性能数据：Soft-RoCE 只用来验功能，eRDMA 只做一次性的按小时校准；
- 元数据依赖 etcd，不自研 Raft：这是刻意的取舍，底座能力由 S1 补上；
- FUSE 与自研失效协议明确不做（工业方案已经成熟，版本前缀也已经消解了问题），不虚报。

13. 砍项清单（v1 → v3，说清楚为什么砍）

v1 内容	处置	理由
增量 1 gate + 增量 2 状态机	**合并为核心 1 闭环**，子集 readyRatio 加盈亏平衡实验	两件事本来就是同一个控制回路；旧指标只报赢家

v1 内容	处置	理由
"内容寻址分块"的措辞	改为路径索引 + CAS 存储层	属于术语硬伤；顺带白赚安全去重
把百万小文件当作唯一/主要负载	降为 A 轨；新增 B 真实分片主臂并前移到步骤 5	2026 年训练的主流是百 MB 分片流式读
在顺序 trace 上对比 W-TinyLFU	改为欠配+倾斜+多 Job 场景，接受阴性结论	顺序场景下三种策略必然趋同
增量 4"权重先起先恢复"	修正：训练恢复必须包含 optimizer state	否则会改变收敛轨迹，属于语义错误
增量 5 普通 P2P	重定位：在 CE 基线上复现 EE 缓存组语义 + Transport 抽象	对标明确，RDMA 也有了挂载点
增量 7 自研版本失效协议	砍掉，用版本前缀目录加一页 ADR 替代	属于人为造深度
FUSE	降为可选 stretch	不重造 JuiceFS/Fluid 的数据面
RDMA/GDS	降为调研/实验臂（Soft-RoCE 加按小时租 eRDMA）	没有硬件，不进主路径
步骤 13 租 GPU	改为双盘 CPU 云主机做存储真值校准	P1 全项目不需要 GPU，省钱

14. 实施步骤（14 步，闸门驱动）

步骤	内容	验收 / 注意点
0 现勘	§4：**4.0 可行性探针（2–3 天，最先做）** + 部署 kind、MinIO、JuiceFS CE、开源 Fluid，跑通 DataLoad/亲和 demo，取证注入内容与时序，产出现勘报告	**双闸门**：探针 A/B 亮绿灯（装置可信）且缺口被证据确认，才能进入核心 1 的闭环叙事；任一条件不满足，就先修装置或收缩叙事，不带病开工；与 P2 现勘同期推进
1 实验装置	tc tbf/netem 限速规程 + drop_caches 规程 + 三轨数据集生成器	`make bench-up`；限速档位可扫点，冷态可重复
2 缓存核心库	manifest 路径索引 + CAS 存储层（流式摘要/refcount）+ 限并发抓取	单测覆盖：并发写同块、tmp-rename 原子性、同内容去重、校验失败重拉
3 Agent 服务化	gRPC（预热/状态）+ /metrics + 启动重扫描 + inotify	指标带 dataset 标签以防基数爆炸；重启恢复要有测试
4 数据面	DaemonSet + hostPath 只读约定 + 预热 CLI	手动预热之后，普通 Pod 可以全量读取
5 Controller + CRD + **真实 dataloader**	reconcile 分发与回填；webdataset/MDS + PyTorch DataLoader 跑通 B 轨	端到端 MVP；informer 重入幂等；**真实负载在此接入，不后置**
6 基准框架与 Fluid 上秤	五组对照自动化（至少先打通 1/2/3/5 组），≥7 次 + CI，一键出图	**第一个简历成果**：包含对手的对照报告

步骤	内容	验收 / 注意点
7 核心 1 闭环	gate + Score + 子集 readyRatio + 候选放置 + 超时降级	**旗舰图**：盈亏平衡曲面 + fanout Pareto；plugin 内无网络调用
8 核心 2 准入预取	C 轨多 Job trace；LRU/LFU/W-TinyLFU + 自适应预取	倾斜场景的差异曲线 + 顺序场景的阴性结论 + 参数敏感性
9 S3 网关	singleflight + io.Pipe 扇出 + AIMD	大对象扇出时内存不涨（用压测证明）
10 Checkpoint sidecar	multipart + manifest 发布点 + 修正后的恢复语义	训练恢复时状态必须完整；恢复时长有拆解
11 P2P 缓存组	哈希环 + peer 回源 + 带宽预算 + Transport 接口	源站出口字节下降；origin 保底生效
12 混沌套件	§8.4 四件套全部通过	恢复时间指标齐备
13 真值校准	两台带独立数据盘的 CPU 云主机 + tc 校准，五组对照全量复跑	用真存储数字替换单机数字，两个版本都保留并说明差异
14 上游回馈	拿着 §6 的数据向 Fluid 提 issue → 讨论 → PR（调度/DataLoad 方向）	至少 1 个带数据的 issue；代码 PR 按维护者的认可程度推进

RDMA 实验臂（§9.1）挂在步骤 11 之后，按兴趣和预算推进，不占关键路径；io_uring（S2）在步骤 8 前后借助 A 轨数据独立推进。

15. 排期与死线（倒推，与 P2 共用日历）

- **2027 年 2 月实习投递包的最低构成**：完成步骤 0–7 和 S1（mini-Raft），并在 Fluid 上提出至少 1 个带测量数据的 issue。也就是说，要有基座加核心 1（五组对照至少跑完三组，再加上盈亏平衡旗舰图）；
- **2027 年 8 月秋招包**：完成步骤 0–11、13、14（issue 有维护者的正式回应，或者 PR 已有实质进展）；
- 砍项顺序（时间不够时从后往前砍）：RDMA 臂 → FUSE → 混沌里的位翻转用例 → P2P 的高级部分（多源调度）→ W-TinyLFU 的 Count-Min 完整版（SlidingWindow LRU 已经足够进简历）；**核心 1、B 轨真实负载、Fluid 对照这三项不砍**；
- 阶段 0 与 P2 现勘共用同一个"使用阶段"，立即启动。

16. 预期成果

交付物：TrainCache 仓库（代码、README、ADR、复现命令）、《P1 对手现勘报告》、五组对照实验报告（包含盈亏平衡曲面与阴性区间）、Grafana 看板、两篇博客（一篇讲调度-预热闭环实测，一篇讲 W-TinyLFU 在什么场景下才有用）、Fluid issue/PR 链接。

简历条目草稿（V1 主版）：

> 设计并实现了一套 K8s 原生的训练数据预热-调度闭环系统：基于 CRD/Operator、SchedulingGate 门控，以及 Job 数据子集的块级就绪度做调度打分与超时降级；存储上采用路径索引加 CAS 内容摘要存储层（内置校验与安全去重）；在 3 节点 kind 集群下与真实的 Fluid/JuiceFS 做了五组对照，以端到端

> makespan 给出预热门控相对自由调度的盈亏边界（源站带宽 × 缓存欠配 × fanout），在重复 epoch、多 Job 共享场景下 makespan 下降 X%，同时量化了它在单次顺序扫描下的阴性区间；在缓存欠配的多租户场景下实测了 W-TinyLFU 与 LRU 的准入差异；相关测量结果以 issue/PR 的形式回馈给 Fluid 社区。